■ AI PREDICTIVE MAINTENANCE PLATFORM

MASTER PRESENTATION, TECHNICAL ARCHITECTURE & DEMO REFERENCE GUIDE

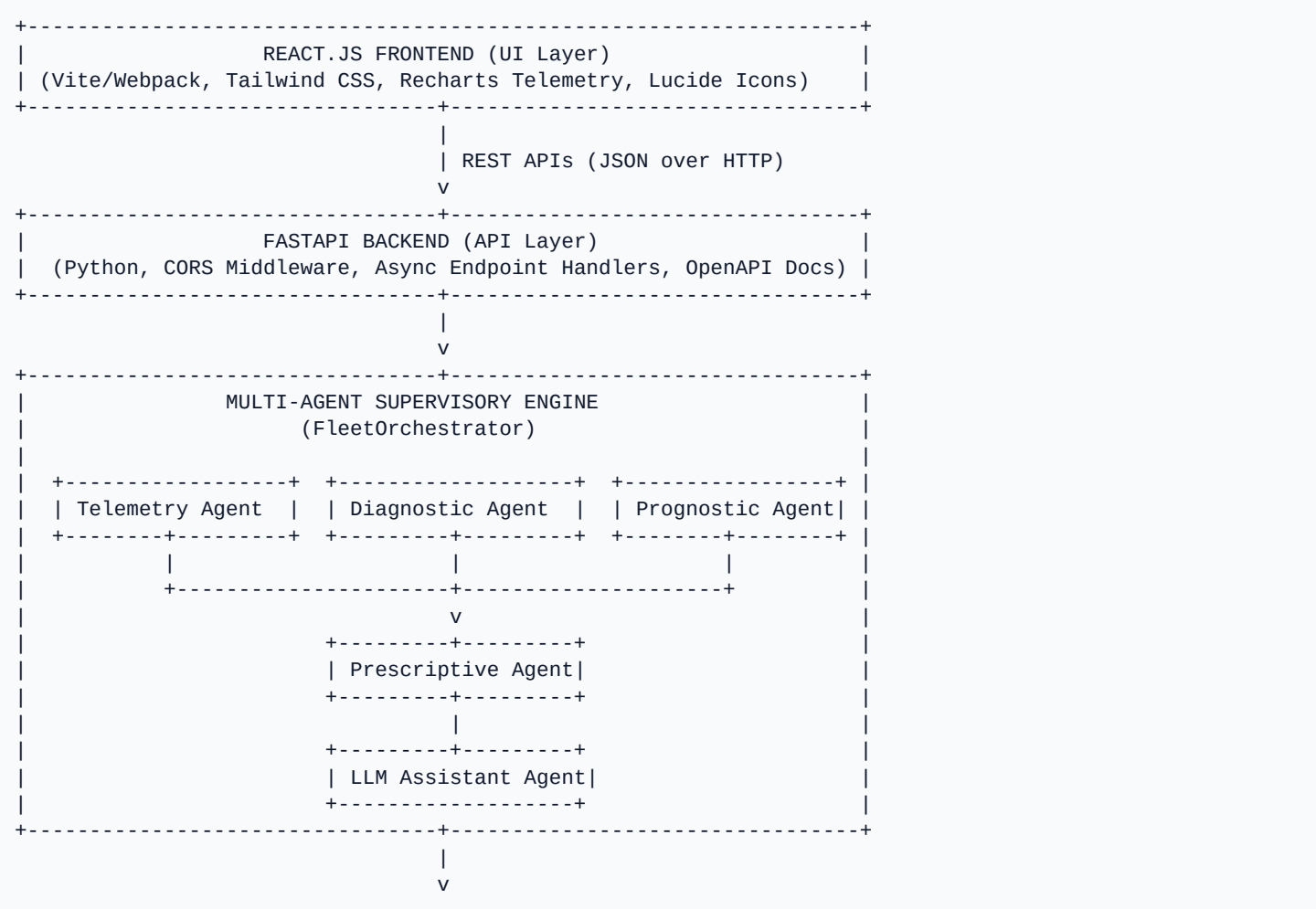
This document is your **complete reference and presentation guide**. It explains **literally everything** happening in your project in plain language, technical architecture, and a step-by-step presentation script so you can understand it deeply and present it confidently tomorrow.

■ Executive Summary (The 30-Second Pitch)

> ***"Unplanned machinery breakdowns cost manufacturing plants millions of dollars in downtime every year. Our system is an Industrial-Grade, Multi-Agent AI Predictive Maintenance (PdM) Platform.***
>
> ***It ingests real-time multi-sensor telemetry (vibration, temperature, pressure, current, RPM), detects anomalies using machine learning, diagnoses exact mechanical faults, forecasts Remaining Useful Life (RUL) in hours, automatically dispatches prescriptive repair work orders with financial ROI calculations, and allows engineers to chat with an AI assistant in natural language."***

■■ Architecture Overview (Decoupled System)

The application follows a modern **Decoupled Architecture**:



```
+--------------------------------+--------------------------------+
|             MACHINE LEARNING & DATA ENGINE                      |
|  (AI4I 2020 Dataset, Isolation Forest, XGBoost RUL, Classifier)   |
+-----------------------------------------------------------------+
```

■ The 5 Autonomous AI Agents & Supervisor

Your backend is powered by a **Multi-Agent Supervisory Architecture**. Each agent has a specific job:

`TelemetryAgent`	Sensor Monitoring & Anomaly Ingestion	Monitors 7 physical sensor channels (Vibration RMS/Kurtosis, Bearing Temp, Pressure, Acoustic Emission, Power, RPM). Uses rolling window feature extraction + **Isolation Forest** ML to detect anomalies.
`DiagnosticAgent`	Root-Cause Fault Classification	When an anomaly is detected, it runs multi-feature classification to pinpoint the exact failure mode (Bearing Fatigue, Hydraulic Leak, Motor Overheating, Tool Wear, Spindle Misalignment).
`PrognosticAgent`	Health Indexing & RUL Forecasting	Runs an **XGBoost / ExtraTrees Regressor** to calculate **Remaining Useful Life (RUL)** in operational hours and computes a dynamic **Machine Health Index (0-100%)**.
`PrescriptiveAgent`	Work Order Dispatch & ROI Optimizer	Generates a structured maintenance work order ticket. Includes priority levels (`CRITICAL_IMMEDIATE`, `HIGH_PRIORITY`), step-by-step repair guides, required spare parts, technician roles, and downtime cost savings.
`LLMAssistantAgent`	Operational Natural Language Chat Assistant	Allows engineers and plant managers to query fleet health, active work orders, and risk schedules using conversational natural language.
`FleetOrchestrator`	Multi-Agent Supervisor	Coordinates data flow between agents, updates live fleet state, and manages the work order ticket lifecycle.

■ Machine Learning Pipeline & Dataset

1. The Dataset (`backend/data/ai4i2020.csv`)

- **Real Industrial Data**: Uses the **AI4I 2020 Predictive Maintenance Dataset** (UCI Machine Learning Repository) containing 10,000 real industrial tool wear & operational readings.
- **Sensor Channels**: Process Temperature, Air Temperature, Rotational Speed (RPM), Torque (Nm), Tool Wear (minutes).

- **Failure Modes**: Tool Wear Failure (TWF), Heat Dissipation Failure (HDF), Power Failure (PWF), Overstrain Failure (OSF), Random Failures (RNF).

2. Trained ML Models

- **Isolation Forest Anomaly Detector**: Unsupervised anomaly detection model trained on healthy baseline telemetry.
- **Prognostic RUL Regressor**: Multi-feature regression model (**XGBoost / ExtraTrees**) predicting RUL hours ($R^2 = 0.8161$, MAE = ~20 hours).
- **Multi-Class Fault Classifier**: Supervised classification model identifying exact failure modes with **95.78% accuracy**.
- **Persistence**: All trained models are serialized into `backend/models/trained_bundle.pkl`. When the server boots up, it loads this bundle in **< 0.1 seconds**!

■ Project Directory Layout

```
Predictive/
■■■ backend/
■   ■■■ app.py                # FastAPI REST application & endpoint handlers
■   ■■■ main.py               # CLI runner & server launcher (--server / --train)
■   ■■■ config.py             # Machine profiles, baseline thresholds, fault modes
■   ■■■ requirements.txt      # Python dependencies (fastapi, uvicorn, scikit-learn, xgboost)
■   ■■■ data/
■   ■   ■■■ ai4i2020.csv      # Real AI4I 2020 industrial dataset (10,000 rows)
■   ■   ■■■ dataset.py        # AI4I loader & rolling window feature engineer
■   ■   ■■■ generator.py      # Synthetic telemetry stream & fault degradation generator
■   ■■■ models/
■   ■   ■■■ anomaly.py        # Isolation Forest anomaly detector
■   ■   ■■■ classifier.py     # Multi-class fault mode classifier
■   ■   ■■■ rul.py            # RUL regression model
■   ■   ■■■ trainer.py        # Automated model training pipeline
■   ■   ■■■ trained_bundle.pkl # Serialized pre-trained model bundle
■   ■■■ agents/               # 5 Autonomous AI agents + supervisor
■   ■■■ tests/                # Backend pytest unit test suite (9 tests passing)
■
■■■ frontend/
■   ■■■ package.json          # React dependencies & Webpack build scripts
■   ■■■ webpack.config.js     # Webpack bundler configuration & API proxy
■   ■■■ src/
■       ■■■ components/
■       ■   ■■■ Header.jsx          # Navbar & API connection status badge
■       ■   ■■■ FleetOverview.jsx   # 5 Metric KPI cards & Machine Health matrix grid
■       ■   ■■■ FaultInjector.jsx   # Interactive simulation fault sandbox
■       ■   ■■■ TelemetryChart.jsx  # Recharts multi-sensor waveform graph
■       ■   ■■■ WorkOrderCenter.jsx # Prescriptive work order ticket center & ROI
■       ■   ■■■ ChatAssistant.jsx   # Operational AI Assistant chat interface
■       ■■■ App.jsx                 # Main layout & tab router
■       ■■■ main.jsx                # React root entrypoint
■       ■■■ index.css               # Industrial dark theme CSS & Grid layout system
```

■ How to Present & Demonstrate Live (Step-by-Step Script)

When you present tomorrow, follow this smooth 6-step demo flow:

Step 1: Start the Backend & Frontend

- Open Terminal 1:

```
cd backend
uvicorn app:app --reload --port 8000
```

- Open Terminal 2:

```
cd frontend
npm start
```

• Open browser at `http://localhost:5173`.

Step 2: Demonstrate Fleet Overview & KPI Matrix

• Point out the **Top 5 KPI Cards**: Total Fleet Machines, Healthy Count, Warning Count, Critical Count, Risk Savings ($).

• Point out the **3-Column Machine Fleet Matrix Grid**: Show the 4 machines (`CNC-MILL-01`, `HYD-PUMP-02`, `IND-COMP-03`, `ROB-ARM-04`), their **Health Index progress bars**, and **Estimated RUL hours**.

Step 3: Trigger a Simulated Fault Injection

• In the **Fault Injection Sandbox** on the left:

1. Select Target Machine: `CNC-MILL-01`.
2. Select Fault Mode: `BEARING_FATIGUE` (Bearing Inner/Outer Race Fatigue).
3. Drag Severity slider to `0.8`.
4. Click **"Inject 1 Step"** or **"Stream 5 Steps"**.

Step 4: Show Real-Time Waveform & Diagnostics

• Watch the **Recharts Sensor Telemetry Chart** update live: Point out how Vibration RMS and Bearing Temperature spike above critical thresholds.

• Point out how the Machine Health Index on `CNC-MILL-01` drops dynamically, and diagnosed fault changes to **BEARING_FATIGUE**.

Step 5: Inspect Prescriptive Work Orders & Financial ROI

• Click the **"Prescriptive Work Orders"** tab.

• Show the automatically generated work order ticket:

• **Priority Badge**: `CRITICAL_IMMEDIATE`

• **Required Spare Parts**: *High-Precision Tapered Roller Bearing Set, Synthetic ISO VG 220 Lubricant*

• **Assigned Technician**: *Senior Vibration Analyst & Mechanical Specialist*

• **Step-by-Step Repair Guide**: 5 structured steps to safely inspect and replace the bearing.

• **Financial Risk ROI**: Show the net savings calculated (e.g. Unplanned Downtime Cost: $6,000 vs. Planned Repair Cost: $850 = **+$5,150 Net Savings**).

Step 6: Demonstrate AI Maintenance Assistant

• Click the **"AI Maintenance Assistant"** tab.

• Click one of the quick sample queries or type:

> *"What is the status of CNC-MILL-01?"*

• Show the AI Assistant replying with health index, RUL forecast, active fault mode, key telemetry readings, and recommended actions.

■ Antigravity Q&A;: Likely Questions & How to Answer Them

****Q1: Why did you decouple into FastAPI + React instead of Streamlit?****

> *"Streamlit is great for rapid prototypes, but it re-executes the entire script on every user interaction, making it hard to scale. By decoupling into a **FastAPI REST backend** and a **React.js single-page application**, we separate heavy ML compute from UI rendering. This provides lower latency, OpenAPI endpoints, and allows mobile or industrial edge systems to consume the same API."*

Q2: How does Remaining Useful Life (RUL) forecasting work?

"We apply rolling window feature engineering (rolling mean, rolling std, rate of change) across historical telemetry readings. An XGBoost/ExtraTrees regressor is trained on historical run-to-failure curves (including the real AI4I 2020 dataset) to predict the exact remaining operating hours before failure."

Q3: How does the business ROI calculation work?

"Each machine profile defines an hourly unplanned downtime cost (e.g., $1,500/hr for CNC Mill, $3,500/hr for Assembly Robot). The Prescriptive Agent compares the cost of catastrophic failure during production vs. scheduled repair time + spare parts cost to calculate net financial savings."

Q4: Do we need to train the model every time the app starts?

"No. The training pipeline serializes the trained models into `backend/models/trained_bundle.pkl`. When FastAPI starts, it loads the pre-trained bundle in less than 100 milliseconds."